

Assignment 2 – Case Study
Stage 4 Tutorial Activities
Object-Oriented Design Decisions
Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	_patient_id must not be blank. _name must contain at least one letter.	get_patient_id(), get_name(), update_details(), get_contact_details()
Practitioner	_practitioner_id, _name and _specialty must not be blank.	get_practitioner_id(), get_name(), get_specialty(), update_details(), update_specialty()
Appointment	_patient must be a Patient. _practitioner must be a Practitioner. _status must be a valid AppointmentStatus. Only scheduled appointments can be rescheduled.	get_appointment_id(), get_patient(), get_practitioner(), get_scheduled_at(), get_status(), change_status(), cancel(), complete(), reschedule()

Activity 2 - Composition or Inheritance?

Appointment and Patient -> □ association Reason: An appointment is linked to a patient, but the patient can exist without the appointment.

Appointment and Practitioner -> association Reason: An appointment is linked to a practitioner, but the practitioner can exist without the appointment.

Doctor and Practitioner (hypothetical) -> □ Inheritance Reason: A Doctor is a specialised type of Practitioner and can inherit the Practitioner's common details, such as name, ID and specialty.

Clinic and Appointment -> □ Composition/association Reason: A clinic can have many appointments, and appointments are separate objects.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

The Appointment class decides because it owns the appointment status rules.

Who validates a patient name?

The Patient class validates its own name when it is created or updated.

Should Appointment execute SQL? Why?

No. Appointment should only manage appointment information and business rules. Database work should be handled by a separate repository or database class.

Should the UI decide whether a status transition is legal?

No. The Appointment class should decide because the rule belongs to the domain object. The UI should only display messages and collect input.

Activity 4 - AI Code Critique

AI generates an Appointment class with public status mutation, SQL inside cancel(), a NotificationManager dependency and inheritance from PatientRecord. Identify at least five design problems and corrections.

Design problem	Correction
Status is public and can be changed directly.	Use an internal <code>_status</code> attribute and control changes through methods.
SQL is placed inside cancel().	Remove SQL and use a separate database or repository class.
The class depends on NotificationManager.	Remove the dependency because notifications are outside the Appointment responsibility.
Appointment inherits from PatientRecord without a clear reason.	Remove the inheritance. Appointment should be its own class.
The class allows invalid status changes.	Add checks and raise <code>InvalidStatusTransitionError</code> .
The constructor does not check object types.	Check that patient is a Patient and practitioner is a Practitioner.
The class mixes UI code with domain logic.	Keep input and menu code separate from the Appointment class.

Exit question

Why can code be object-oriented syntactically but still have poor object-oriented design?

Code can be object-oriented syntactically because it uses classes and objects, but still have poor object-oriented design if the responsibilities are placed in the wrong classes. For example, an Appointment class that runs SQL, sends notifications, controls the user interface and allows public status changes has too many responsibilities. Good object-oriented design keeps each class focused, protects its data and keeps business rules inside the correct class.